



Practical No 4

Aim

Inheritance/Access Modifiers Example using Typescript

Apparatus / Software Required

- Visual Studio Code / Any Code Editor
- TypeScript
- Node.js
- Web Browser

Theory

TypeScript supports Object-Oriented Programming (OOP) concepts such as **classes, objects, inheritance, interfaces, access modifiers, constructors, and polymorphism**.

In this practical, TypeScript is used to understand how classes can share properties and methods through inheritance and how access modifiers can control the visibility of class members.

Concepts Required

1. TypeScript

TypeScript is a superset of JavaScript developed by Microsoft. It adds static typing and supports Object-Oriented Programming concepts.

TypeScript helps developers write code that is more structured, readable, maintainable, and easier to debug.

Features

- Static Type Checking
- Object-Oriented Programming Support
- Better Error Detection
- Classes and Objects
- Interfaces
- Access Modifiers
- Inheritance
- Improved Code Readability
- Easy Maintenance
-

Example

```
let studentName: string = "Manasi";  
let rollNo: number = 101;
```

2. Class

A class is a blueprint for creating objects. It defines the properties and methods that an object can have.



Example

```
class Student {  
    name: string;  
    rollNo: number;  
  
    displayDetails(): void {  
        console.log(this.name);  
        console.log(this.rollNo);  
    }  
}
```

Here, Student is a class containing properties and a method.

3. Object

An object is an instance of a class. It is created using the new keyword.

Example

```
let student = new Student();
```

```
student.name = "Manasi";  
student.rollNo = 101;
```

```
student.displayDetails();
```

The object student can access the properties and methods defined in the Student class.

4. Constructor

A constructor is a special method that is automatically called when an object of a class is created. It is mainly used to initialize the properties of an object.

Example

```
class Student {  
    name: string;  
    rollNo: number;  
  
    constructor(name: string, rollNo: number) {  
        this.name = name;  
        this.rollNo = rollNo;  
    }  
}
```

An object can be created as:

```
let student = new Student("Manasi", 101);
```

The constructor automatically initializes name and rollNo.

5. Access Modifiers

Access modifiers control the visibility and accessibility of class members.

TypeScript provides three main access modifiers:

- public
- private
- protected



Public

A public member can be accessed from anywhere.

```
class Student {  
    public name: string = "Manasi";  
}
```

It can be accessed outside the class:

```
let s = new Student();  
console.log(s.name);
```

Private

A private member can only be accessed inside the class where it is declared.

```
class Student {  
    private rollNo: number = 101;  
  
    displayRollNo(): void {  
        console.log(this.rollNo);  
    }  
}
```

The rollNo cannot be directly accessed outside the class.

Protected

A protected member can be accessed inside the class and its derived classes.

```
class Person {  
    protected name: string = "Bhaskar";  
}  
  
class Student extends Person {  
    displayName(): void {  
        console.log(this.name);  
    }  
}
```

Here, Student can access the protected name property because it inherits from Person.

6. Inheritance

Inheritance is an Object-Oriented Programming concept in which one class acquires the properties and methods of another class.

In TypeScript, inheritance is implemented using the extends keyword.

The class being inherited is called the **parent/base class**, while the class that inherits it is called the **child/derived class**.

Example

```
class Person {  
    name: string = "Manasi";  
  
    displayName(): void {  
        console.log(this.name);  
    }  
}
```



```
}  
}  
  
class Student extends Person {  
    rollNo: number = 101;  
  
    displayStudent(): void {  
        console.log(this.name);  
        console.log(this.rollNo);  
    }  
}
```

Here, Student inherits the name property and displayName() method from Person.

7. Types of Inheritance

Inheritance can be classified into different types based on the relationship between parent and child classes.

a) Single Inheritance

In single inheritance, one child class inherits from one parent class.

```
class Person {  
    name: string = "Manasi";  
}  
  
class Student extends Person {  
    rollNo: number = 101;  
}
```

Structure:

Person



Student

Here, Student inherits from Person.

b) Multilevel Inheritance

In multilevel inheritance, a class inherits from another derived class, forming a chain of inheritance.

```
class Person {  
    name: string = "Manasi";  
}  
  
class Student extends Person {  
    rollNo: number = 101;  
}  
  
class Result extends Student {  
    marks: number = 85;  
}
```



Structure:

Person



Student



Result

Here, Result inherits from Student, and Student inherits from Person.

c) Hierarchical Inheritance

In hierarchical inheritance, multiple child classes inherit from the same parent class.

```
class Person {  
    name: string = "Manasi";  
}  
  
class Student extends Person {  
    rollNo: number = 101;  
}  
  
class Teacher extends Person {  
    subject: string = "TypeScript";  
}
```

Structure:

Person
/ \

Student Teacher

Here, both Student and Teacher inherit from Person.

8. Method Overriding

Method overriding occurs when a child class provides its own implementation of a method that already exists in the parent class.

Example

```
class Person {  
    display(): void {  
        console.log("This is a person");  
    }  
}  
  
class Student extends Person {  
    display(): void {  
        console.log("This is a student");  
    }  
}
```

When display() is called using a Student object, the child class implementation is executed. Method overriding allows a child class to provide behavior specific to its requirements.



9. Interface

An **interface** in TypeScript is used to define the structure or blueprint of an object or class. It specifies the properties and methods that an object or class should contain.

Interfaces help in achieving:

- Code consistency
- Type safety
- Better code organization
- Standard structure

An interface can contain property and method declarations, but it does not provide their implementation.

Example

```
interface Student {  
    name: string;  
    rollNo: number;  
  
    displayDetails(): void;  
}
```

A class can implement an interface using the implements keyword.

```
class StudentDetails implements Student {  
  
    name: string;  
    rollNo: number;  
  
    constructor(name: string, rollNo: number) {  
        this.name = name;  
        this.rollNo = rollNo;  
    }  
  
    displayDetails(): void {  
        console.log("Name: " + this.name);  
        console.log("Roll No: " + this.rollNo);  
    }  
}
```

```
let student = new StudentDetails("Manasi", 101);  
student.displayDetails();
```

Here, StudentDetails must provide the displayDetails() method declared in the Student interface.



Shri Yashwantrao Bhonsale Education Society's

YASHWANTRAO BHONSALE INSTITUTE OF TECHNOLOGY

(DTE CODE : 3470) (MSBTE Code : 1742)

Approved by AICTE, DTE & Affiliated to Mumbai University & MSBTE Mumbai
(NBA Accredited ME, CE, EE Diploma Programs)

Conclusion:
